



Universidade Federal de Viçosa – Campus
UFV-Florestal
Ciência da Computação – Engenharia de Software II
Projeto Integrador 2025

Modelo de Classes - sprint 04

Designer de Sistemas

Equipe 03 - Jogo Desconecta

Ana Lúcia de Souza - 5084

Florestal

2025

Sumário

Back-end.....	3
Classes de Entidade (Entity):.....	3
Classes de Controle:.....	4
Classes de acesso ao Banco de Dados:.....	5
Relacionamento entre as classes.....	5
Front-end.....	6

Back-end

A figura 1 mostra o Diagrama de Classes para os Casos de Uso a ser codificado na Sprint 04, isto é, o **CSU07: Jogo Caça-Palavras**. Observe que esse diagrama utiliza a categorização BCE (Boundary, Control, Entity), a qual implica que cada objeto é especialista em realizar um dos três tipos de tarefa, a saber: comunicar-se com atores (fronteira), manter as informações (entidade) ou coordenar a realização de um caso de uso (controle).

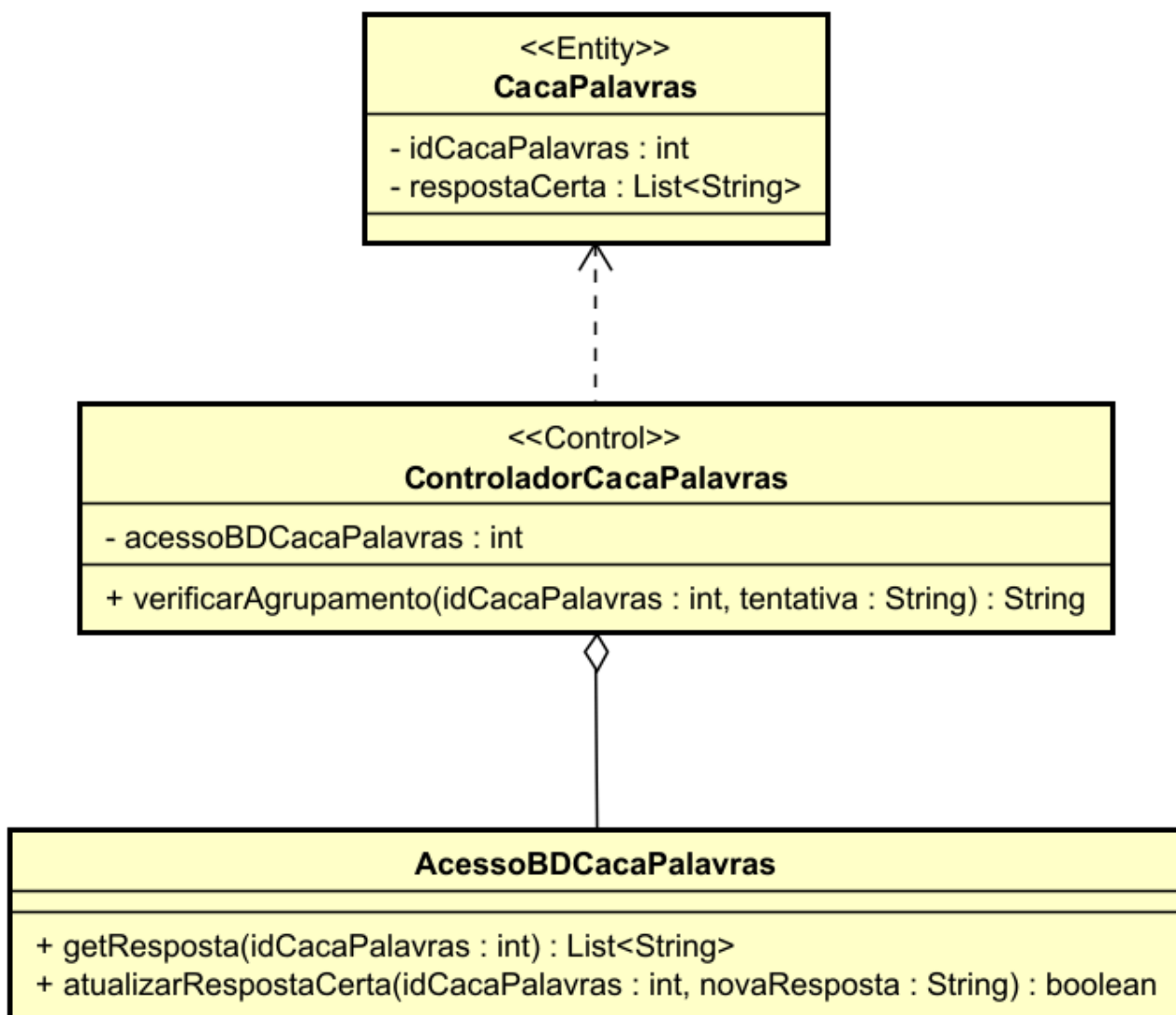


Figura 1. Diagrama de Classes para CSU07.

Classes de Entidade (Entity):

A seguir será explicada a única classe de entidade deste caso de uso:

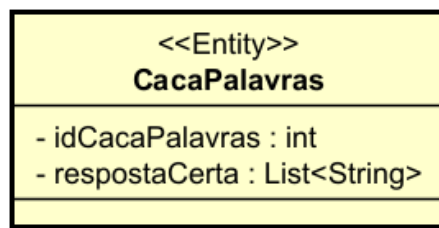


Figura 2. Classe CacaPalavras.

- CacaPalavras:** Essa classe possui dois argumentos privados como mostrado na Figura 2. O atributo “**idCacaPalavras**”, é o identificador do jogo Caça-Palavras, assim, se, por exemplo, as ilhas Dadolândia, Ciências e Matemática possuem esse tipo de desafio, então cada um deles deve ter um identificador diferente a fim de diferenciá-los. Já o atributo “**respostaCerta**” é uma lista de strings as quais são as responsáveis por conter as respostas certas associadas ao jogo Caça-Palavras em questão. Aqui, foi escolhido usar uma lista para armazenar as respostas certas, ou seja, as palavras corretas do desafio ao invés de se escolher um número de atributos como foi feito no caso de uso anterior (CSU 06: Jogo conecta). Observe que, por isso, internamente no banco de dados, haverá outra tabela, não sendo esta uma tabela de entidade, e sendo apenas a tabela que efetivamente faz o relacionamento entre a classe em questão e a lista de strings. Caso a presença desta nova tabela se constituir como um problema de desempenho no banco de dados, ou seja, se o número de consultas já realizadas estiver causando lentidão na resposta do banco de dados, pode-se usar da mesma estratégia feita no caso de uso anterior, ou seja, deve-se ter vários atributos responsáveis por guardar as respostas corretas e seu número exato deve ser igual ao número de palavras a serem encontradas no desafio. Os métodos getter e setters podem ser implementados conforme a necessidade do programador.

Classes de Controle:

A classe “**ControladorCacaPalavras**”, possui o atributo privado **acessoBDCacaPalavras**, que será criado pelo construtor dessa classe. Nesta classe existe o seguinte método:

- verificarAgrupamento(idCacaPalavras: int, tentativa: String) : String:** Esse método tem como objetivo verificar se uma resposta fornecida pelo aluno está correta ou não, ou seja, se a string formada no front-end pelas posições de cada letra que compõe uma palavra que o aluno tenta acertar no jogo Caça-Palavras, se

constitui como uma das strings que representa a resposta certa para esse desafio específico. Para isso, esse método deve utilizar o método **getResposta()** da classe **AcessoBDCacaPalavras**. Desse modo, depois que **getResposta()** retornar a lista das strings detentoras das respostas do desafio, o método **verificarAgrupamento()** deve comparar a string - vinda do front - com as strings presentes no banco de dados. Havendo um “casamento” entre a string formada pelo aluno e uma das strings existentes no banco, o método deve retornar uma mensagem indicando que a resposta está correta, caso contrário deve retornar uma mensagem indicando que está errada. Observe que mesmo que este método retorne uma mensagem positiva, isso não significa que o desafio foi concluído, pois é necessário que todos os agrupamentos tenham sido feitos de forma correta, ou seja, que o aluno tenha encontrado todas as palavras no caça-palavras. Sendo assim, também será necessário verificar se a lista de palavras a serem encontradas já chegou ao fim. Isso deve ser feito no front-end “riscando-se” as palavras que já foram encontradas pelo aluno. Assim, o jogo termina quando todas as palavras forem riscadas, ou seja, quando todas as palavras forem encontradas na matriz do jogo.

Classes de acesso ao Banco de Dados:

A classe **AcessoBDCacaPalavras** será a responsável pelo acesso e manipulação do banco de dados referente à tabela **CacaPalavras**. Seus métodos serão descritos a seguir:

- **getResposta(idCacaPalavras: int): List<String>**: Esse método recebe o id do Caça Palavras e consulta o banco de dados para obter uma lista de respostas certas, ou seja, o conjunto das respostas corretas referente a este desafio.
- **atualizarRespostaCerta(idCacaPalavras : int, novaResposta : String) : boolean**: Esse método possui apenas o objetivo de encapsular a lógica para alterar uma das respostas do desafio presentes no banco de dados, caso isso seja necessário em algum momento. Não sendo, portanto, um método necessário para a lógica do jogo, apenas útil aos desenvolvedores.

Relacionamento entre as classes

Observe que a classe **ControladorCacaPalavras** apresenta um relacionamento de agregação com **AcessoBDCacaPalavras**, sendo a primeira o objeto "todo" e a segunda o objeto "parte". A agregação representa que o controlador "tem um" acesso ao banco de

dados para cumprir sua função. Em relação a **ControladorCacaPalavras**, há uma relação de dependência com a classe **CacaPalavras**, pois o método “verificarAgrupamento()” usa o “idCacaPalavras” como parâmetro.

Front-end

Para a modelagem do front-end, foi utilizado o diagrama de máquina de estados conforme apresentado na figura 3. Nesse diagrama, **os estados representam as telas ou popUps e as transições representam os botões.**

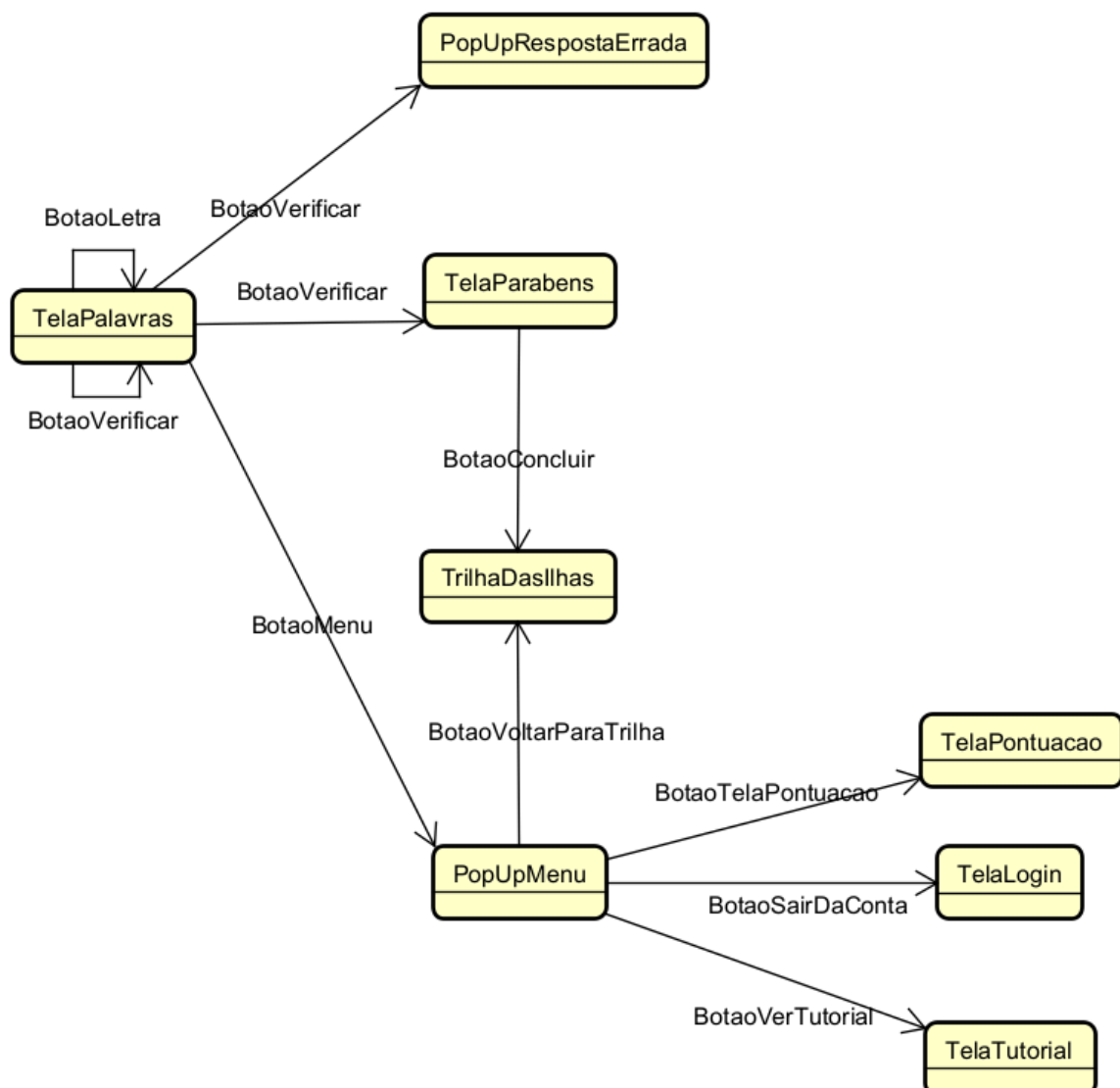


Figura 3. Modelagem front-end.

A “TelaPalavras” do desafio Caça-Palavras representa a Tela a qual deve conter a matriz do caça palavras e as palavras a serem descobertas pelo aluno. A ação do aluno

de selecionar as letras as quais compõem uma palavra é feita pelo “BotaoLetra”, ou seja, cada letra presente na matriz que compõe o Caça-Palavras é um botão clicável. Depois que o aluno selecionar as letras que formam uma palavra a qual julga correta, ele deve clicar no “BotaoVerificar” para que o sistema possa validar sua jogada ou não. Caso a palavra selecionada não esteja correta, a ação do “BotaoVerificar”, levará o aluno para um popUp que indica que os elementos selecionados não estão corretos. Esse popUp deve desaparecer após certo tempo e o aluno será levado novamente para a tela do caça-palavras. Como essa ação não usa um botão, a transição entre o popUp de volta a tela não foi representada por uma transição no diagrama. Quando o aluno realizar o último agrupamento de letras correto, ou seja, encontrar a última palavra na lista, a ação do “BotaoVerificar” o levará a “TelaParabens”, um marco para a conclusão do desafio. Desse modo, ao clicar em “BotaoConcluir”, o aluno é levado de volta para a trilha de fases (Ilhas).

No canto superior direito há um botão, o “BotaoMenu”, que se expande para uma barra de navegação, ou um popUp (“PopUpMenu”). Nela, há a opção de o aluno voltar para a trilha (TelaDasIlhas), ir para a tela de pontuação (TelaPontuacao), sair da conta (TelaLogin) e ver tutorial (TelaTutorial).



Figura 4. Prototipagem de TelaPalavras.



Figura 5. Prototipagem de TelaParabens.

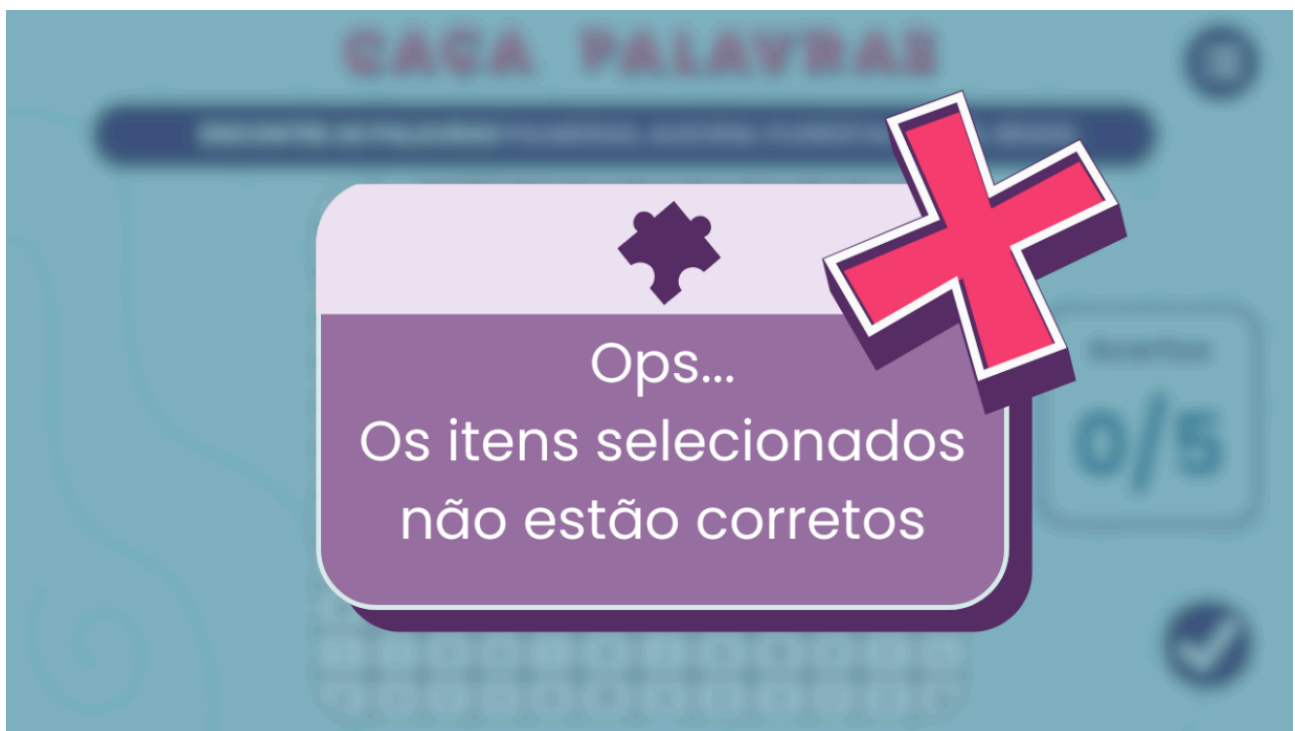


Figura 6. Prototipagem de PopUpRespostaErrada.

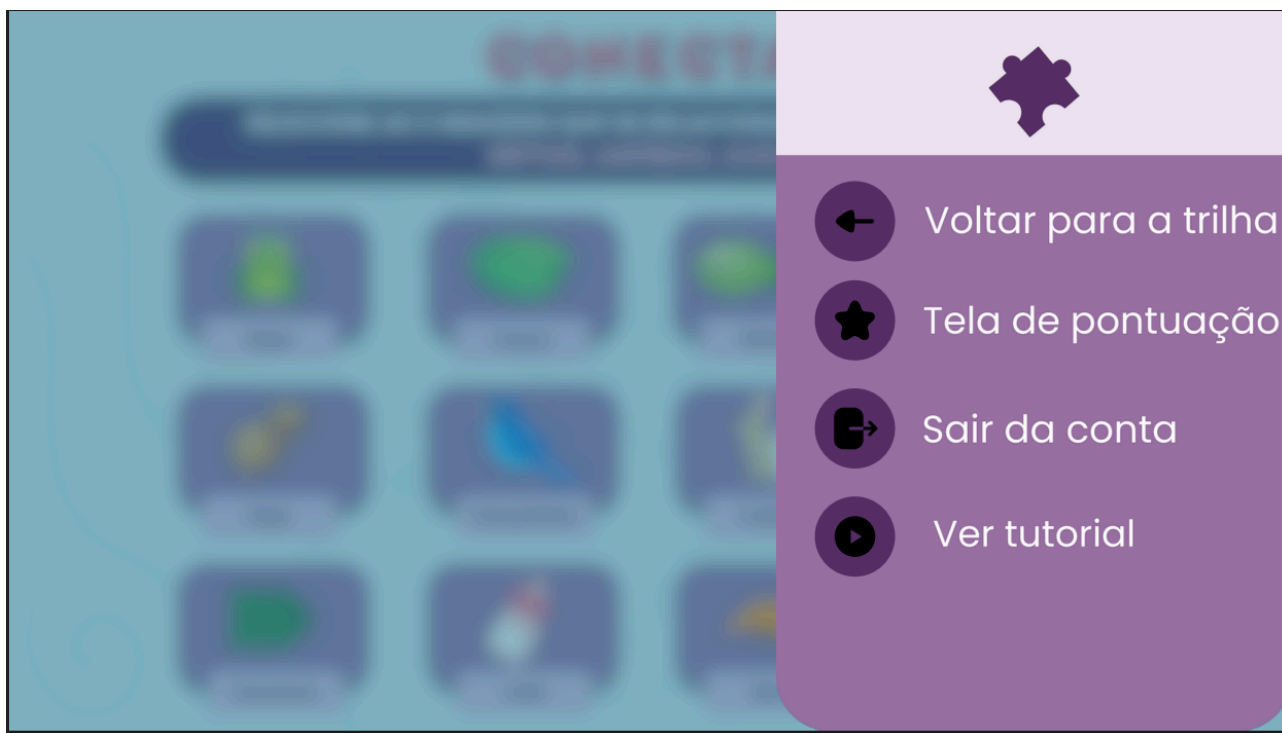


Figura 7. Prototipagem de PopUpMenu.

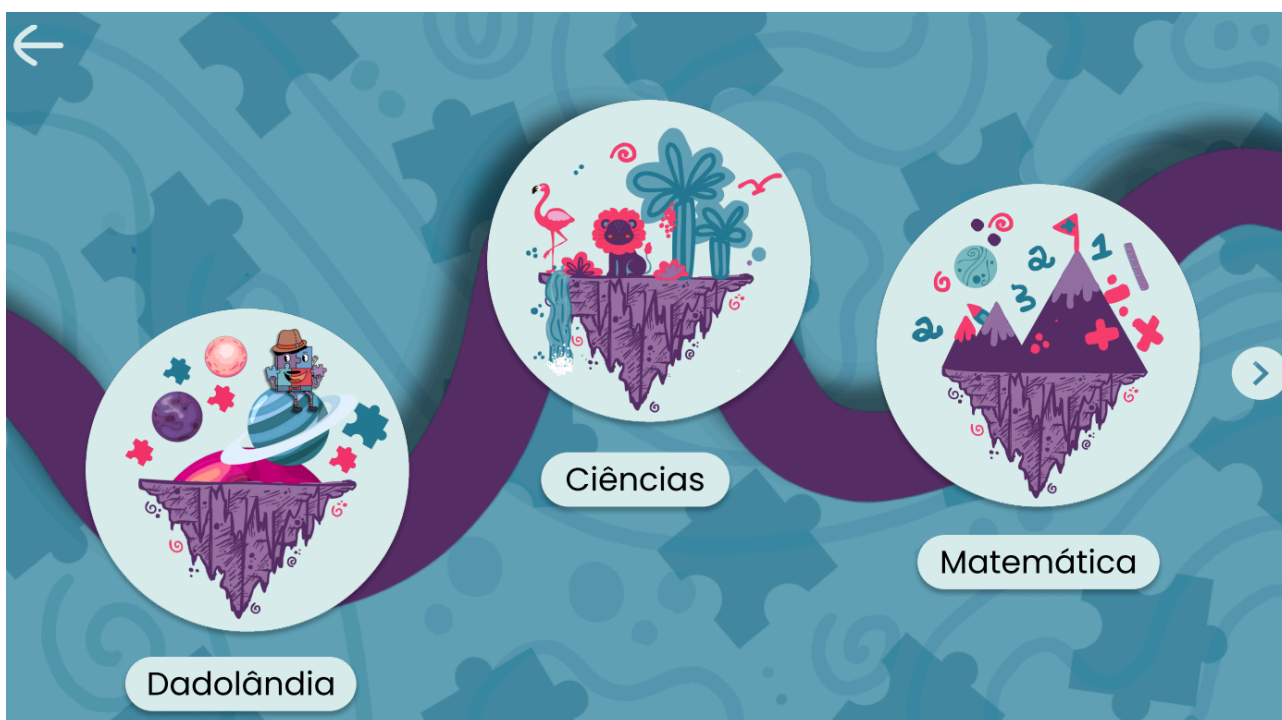


Figura 8. Prototipagem de TrilhaDasIlhas.



Figura 9. Prototipagem de TelaLogin.

OBS: As demais telas não foram incluídas pois ainda não foram prototipadas.